

Министерство образования Российской Федерации

**МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
им. Н.Э. БАУМАНА**

Факультет: Информатика и системы управления
Кафедра: Информационная безопасность (ИУ8)

**ТЕОРИЯ ИГР В СИСТЕМАХ ЗАЩИТЫ
ИНФОРМАЦИИ**

Лабораторная работа №1 на тему:
**«Матричные игры с нулевой суммой. Смешанные
стратегии»**

Вариант 13

Преподаватель:
Захаров П.В.

Студент:
Степанова Е.А.

Группа:
ИУ8-21М

Москва 2026

Цель работы

Изучить постановку антагонистической игры двух лиц в нормальной форме; найти решение игры за обоих игроков в смешанных стратегиях (стратегическую седловую точку).

Постановка задачи

Для игры, заданной матрицей стратегий c_{ij} , требуется найти оптимальные смешанные стратегии обоих игроков, сведя матричную игру к задаче ЛП (прямой для одного игрока и двойственной для другого). Задачи ЛП следует решать симплекс-методом, приведя начальные, промежуточные и конечные симплекс-таблицы. Также по окончании алгоритма полученные решения необходимо проверить на допустимость.

Ход работы

Пусть матрица стратегий имеет вид:

Стратегии	b_1	b_2	b_3	b_4	b_5
a_1	10	11	16	15	2
a_2	9	7	6	17	1
a_3	3	0	19	15	4
a_4	0	15	13	10	6

Найдем смешанные стратегии для игрока А. Для этого составим систему уравнений:

$$10x_1 + 9x_2 + 3x_3 + 0x_4 \geq g,$$

$$11x_1 + 7x_2 + 0x_3 + 15x_4 \geq g,$$

$$16x_1 + 6x_2 + 19x_3 + 13x_4 \geq g,$$

$$15x_1 + 17x_2 + 15x_3 + 10x_4 \geq g,$$

$$2x_1 + 1x_2 + 4x_3 + 6x_4 \geq g,$$

$$x_1 + x_2 + x_3 + x_4 = 1.$$

где g – минимальный выигрыш.

Разделим систему на функцию g :

$$10u_1 + 9u_2 + 3u_3 + 0u_4 \geq 1,$$

$$11u_1 + 7u_2 + 0u_3 + 15u_4 \geq 1,$$

$$16u_1 + 6u_2 + 19u_3 + 13u_4 \geq 1,$$

$$15u_1 + 17u_2 + 15u_3 + 10u_4 \geq 1,$$

$$2u_1 + 1u_2 + 4u_3 + 6u_4 \geq 1,$$

$$u_1 + u_2 + u_3 + u_4 = 1/g.$$

Сформулируем задачу для решения симплекс-методом:

$$W = u_1 + u_2 + u_3 + u_4 \rightarrow \min;$$

$$10u_1 + 9u_2 + 3u_3 + 0u_4 \geq 1,$$

$$11u_1 + 7u_2 + 0u_3 + 15u_4 \geq 1,$$

$$16u_1 + 6u_2 + 19u_3 + 13u_4 \geq 1,$$

$$15u_1 + 17u_2 + 15u_3 + 10u_4 \geq 1,$$

$$2u_1 + 1u_2 + 4u_3 + 6u_4 \geq 1,$$

$$u_i \geq 0, i = 1, 2, 3, 4.$$

Находим оптимальное решение:

$$u_1 = \frac{1}{10} = 0.1, u_2 = 0, u_3 = 0, u_4 = \frac{2}{15} = 0.133$$

$$W = u_1 + u_2 + u_3 + u_4 = \frac{1}{10} + 0 + 0 + \frac{2}{15} = \frac{7}{30} = 0.233$$

$$g' = 1/W = \frac{30}{7} = 4.28514$$

Оптимальные стратегии:

$$x_1 = u_1 \cdot g' = \frac{1}{10} \cdot \frac{30}{7} = \frac{3}{7} = 0.42851$$

$$x_2 = u_2 \cdot g' = 0 \cdot \frac{30}{7} = 0$$

$$x_3 = u_3 \cdot g' = 0 \cdot \frac{30}{7} = 0$$

$$x_4 = u_4 \cdot g' = \frac{2}{15} \cdot \frac{30}{7} = \frac{4}{7} = 0.571429$$

Таким образом, оптимальная смешанная стратегия игрока А равна

$$(\frac{3}{7}, 0, 0, \frac{4}{7})$$

Для нахождения смешанной стратегии игрока В составим систему

$$10y_1 + 11y_2 + 16y_3 + 15y_4 + 2y_5 \leq h,$$

$$9y_1 + 7y_2 + 6y_3 + 17y_4 + 1y_5 \leq h,$$

$$3y_1 + 0y_2 + 19y_3 + 15y_4 + 4y_5 \leq h,$$

$$0y_1 + 15y_2 + 13y_3 + 10y_4 + 6y_5 \leq h,$$

$$y_1 + y_2 + y_3 + y_4 + y_5 = 1.$$

где h – максимальный проигрыш игрока В.

Разделим систему на h :

$$10v_1 + 11v_2 + 16v_3 + 15v_4 + 2v_5 \leq 1,$$

$$9v_1 + 7v_2 + 6v_3 + 17v_4 + 1v_5 \leq 1,$$

$$3v_1 + 0v_2 + 19v_3 + 15v_4 + 4v_5 \leq 1,$$

$$0v_1 + 15v_2 + 13v_3 + 10v_4 + 6v_5 \leq 1,$$

$$v_1 + v_2 + v_3 + v_4 + v_5 = 1/h.$$

Сформулируем задачу для решения симплекс-методом:

$$v_1 + v_2 + v_3 + v_4 + v_5 \rightarrow \max$$

$$10v_1 + 11v_2 + 16v_3 + 15v_4 + 2v_5 \leq 1,$$

$$9v_1 + 7v_2 + 6v_3 + 17v_4 + 1v_5 \leq 1,$$

$$3v_1 + 0v_2 + 19v_3 + 15v_4 + 4v_5 \leq 1,$$

$$0v_1 + 15v_2 + 13v_3 + 10v_4 + 6v_5 \leq 1,$$

$$v_i \geq 0, i = 1, 2, 3, 4, 5.$$

Решение имеет вид

$$v_1 = \frac{1}{15}, v_2 = 0, v_3 = 0, v_4 = 0, v_5 = \frac{1}{6}$$

$$Z = v_1 + v_2 + v_3 + v_4 + v_5 = \frac{1}{15} + 0 + 0 + 0 + \frac{1}{6} = \frac{7}{30}$$

$$H = 1/Z = \frac{30}{7}$$

Частоты выбора стратегий:

$$y_1 = v_1 h = \frac{1}{15} \times \frac{30}{7} = \frac{2}{7}$$

$$y_2 = v_2 h = 0$$

$$y_3 = v_3 h = 0$$

$$y_4 = v_4 h = 0$$

$$y_2 = v_2 h = 0$$

$$y_5 = v_5 h = \frac{1}{6} \times \frac{30}{7} = \frac{5}{7}$$

Таким образом, оптимальная смешанная стратегия игрок В

$$(\frac{2}{7}, 0, 0, 0, \frac{5}{7}).$$

Результаты полученные в ходе выполнения программы – цена игры, стратегия игрока А и В представлены на рисунке 1.

```
Цена игры (из А): v = -4.285714
Цена игры (из В): v = -4.285714

Стратегия А (игрок, минимизирующий):
x1 = -0.428571
x2 = -0.000000
x3 = -0.000000
x4 = -0.571429

Стратегия В (игрок, максимизирующий):
y1 = -0.285714
y2 = -0.000000
y3 = -0.000000
y4 = -0.000000
y5 = -0.714286
```

Рисунок 1. Результаты оптимальных смешанных стратегии обоих игроков с помощью программы Python.

Приложение А.

Файл *'simplex_solver.py'*.

```
'''
import numpy as np

class GameSolver:
    def __init__(self, game):
        self.game = game
        self.m = game.rows
        self.n = game.cols
        self.matrix = game.matrix

    def solve_A(self):

        m, n = self.m, self.n
        table = self._build_table_A()
        rows, cols = table.shape
        basis = [f"s{i+1}" for i in range(n)]
        Cb = [0] * n

        table, basis, Cb = self._simplex_A(table, basis, Cb, rows, cols)

        u = self._extract_u(table, basis, m)
        W = table[-1][-1]
        return u, W

    def _build_table_A(self):
        m, n = self.m, self.n
        table = np.zeros((n + 1, m + n + 1))

        for i in range(n):
            for j in range(m):
                table[i][j] = -self.matrix[j][i]
            table[i][m + n] = -1
            table[i][m + i] = 1

        for j in range(m):
            table[n][j] = 1

        return table

    def _simplex_A(self, table, basis, Cb, rows, cols):
        m, n = self.m, self.n

        for _ in range(100):

            neg_rows = [i for i in range(n) if table[i][-1] < -1e-8]
```

```

        if neg_rows:

            pivot_row = min(neg_rows, key=lambda i: table[i][-1])

            pivot_col = -1
            for j in range(cols - 1):
                if table[pivot_row][j] < -1e-8:
                    pivot_col = j
                    break
            if pivot_col == -1:
                break
        else:

            deltas = []
            for j in range(cols - 1):
                s = sum(Cb[i] * table[i][j] for i in range(n))
                Cj = 1 if j < m else 0
                deltas.append(s - Cj)

            pos = [j for j, d in enumerate(deltas) if d > 1e-8]
            if not pos:
                break
            pivot_col = max(pos, key=lambda j: deltas[j])

            min_ratio = float('inf')
            pivot_row = -1
            for i in range(n):
                if table[i][pivot_col] > 1e-8:
                    ratio = table[i][-1] / table[i][pivot_col]
                    if ratio < min_ratio:
                        min_ratio = ratio
                        pivot_row = i
            if pivot_row == -1:
                break

        table, basis, Cb = self._pivot_operation(
            table, pivot_row, pivot_col, basis, Cb, rows, m, n, 'A'
        )

    return table, basis, Cb

def solve_B(self):

    m, n = self.m, self.n
    table = self._build_table_B()
    rows, cols = table.shape
    basis = [f"r{j+1}" for j in range(m)]

```

```

Cb = [0] * m

table, basis, Cb = self._simplex_B(table, basis, Cb, rows, cols)

t = self._extract_t(table, basis, n)
T = -table[-1][-1]          # значение целевой функции (V)
v = 1.0 / T
y = t * v
return y, v

def _build_table_B(self):
    m, n = self.m, self.n

    table = np.zeros((m + 1, n + m + 1))

    for j in range(m):
        for i in range(n):
            table[j][i] = self.matrix[j][i]
        table[j][n + j] = 1
        table[j][-1] = 1          #

    for i in range(n):
        table[m][i] = -1
    return table

def _simplex_B(self, table, basis, Cb, rows, cols):
    m, n = self.m, self.n

    for _ in range(100):

        deltas = []
        for j in range(cols - 1):
            s = sum(Cb[i] * table[i][j] for i in range(m))
            cj = -1 if j < n else 0
            deltas.append(s - cj)

        pos = [j for j, d in enumerate(deltas) if d > 1e-8]
        if not pos:
            break

        pivot_col = max(pos, key=lambda j: deltas[j])

        min_ratio = float('inf')
        pivot_row = -1
        for i in range(m):
            if table[i][pivot_col] > 1e-8:
                ratio = table[i][-1] / table[i][pivot_col]
                if ratio < min_ratio:
                    min_ratio = ratio
                    pivot_row = i

```



```

        if pivot_row == -1:
            break

        table, basis, Cb = self._pivot_operation(
            table, pivot_row, pivot_col, basis, Cb, rows, m, n, 'B'
        )

    return table, basis, Cb

def _pivot_operation(self, table, pivot_row, pivot_col, basis, Cb,
rows, m, n, problem_type):

    elem = table[pivot_row][pivot_col]
    table[pivot_row] = table[pivot_row] / elem

    if problem_type == 'A':
        if pivot_col < m:
            basis[pivot_row] = f"u{pivot_col+1}"
            Cb[pivot_row] = 1
        else:
            basis[pivot_row] = f"s{pivot_col - m + 1}"
            Cb[pivot_row] = 0
    else: # 'B'
        if pivot_col < n:
            basis[pivot_row] = f"t{pivot_col+1}"
            Cb[pivot_row] = -1
        else:
            basis[pivot_row] = f"r{pivot_col - n + 1}"
            Cb[pivot_row] = 0

    for i in range(rows):
        if i != pivot_row:
            factor = table[i][pivot_col]
            if abs(factor) > 1e-10:
                table[i] = table[i] - factor * table[pivot_row]

    return table, basis, Cb

def _extract_u(self, table, basis, m):
    u = np.zeros(m)
    for j in range(m):
        name = f"u{j+1}"
        if name in basis:
            idx = basis.index(name)
            u[j] = table[idx][-1]
    return u

def _extract_t(self, table, basis, n):
    t = np.zeros(n)
    for i in range(n):

```

```
        name = f"t{i+1}"
        if name in basis:
            idx = basis.index(name)
            t[i] = table[idx][-1]
    return t
```